

# I/O from Python to the operating system

The call you write and the call the operating system runs are not the same call. `fflush()` flushes only the C library's buffers, and the kernel buffers need a second flush [S3]. Three of the four boundaries below your code have a tool that watches them [V10] [S1] [V30]. No tool in this lecture watches the fourth [V5].

---

**Ondrej (Ondra) Krajicek**

me@ondrejkrjicek.com · [www.linkedin.com/in/OndrejKrajicek](https://www.linkedin.com/in/OndrejKrajicek)

Built 2026-09-14 11:24 UTC

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

# Roadmap

|   |                                             |                    |
|---|---------------------------------------------|--------------------|
| 1 | <b>What does one byte cost?</b>             | A stopwatch.       |
| 2 | <b>Where does the difference come from?</b> | A tracer.          |
| 3 | <b>It is not one buffer — it is a stack</b> | A diagram.         |
| 4 | <b>The same write, at three levels</b>      | A program.         |
| 5 | <b>Two operating systems, side by side</b>  | Two traces.        |
| 6 | <b>What we have not looked at</b>           | One open question. |

# Takeaways

- |   |                                            |                                            |
|---|--------------------------------------------|--------------------------------------------|
| 1 | <b>Buffering pays on adjacency</b>         | A seek forces the buffer out.              |
| 2 | <b>A library call is not a system call</b> | Several levels sit beneath yours [S3].     |
| 3 | <b>The boundary is observable</b>          | A tracer shows what your line became [S1]. |
| 4 | <b>The platforms differ by design</b>      | Each nominates a different level [V8].     |

## SCOPE

# What this lecture covers

- 1 Why two programs differ
- 2 When a buffer helps
- 3 The four levels
- 4 Reading a trace

## SCOPE

# What this lecture leaves out

- 5 When a write is safe
- 6 Memory mapping
- 7 Operating the tools
- 8 Asynchronous I/O

PART I

# What does one byte cost?

---

Two programs. The same mebibyte. Predict the ratio before we run them.

# What does one byte cost?

Two programs write the same mebibyte to a file. That is 1,048,576 bytes. One uses a buffer. One does not.

```
# A: unbuffered — one call per byte
with open("a.out", "wb", buffering=0) as f:
    for i in range(1024 * 1024):
        f.write(b"x")

# B: buffered — the default
with open("b.out", "wb") as f:
    for i in range(1024 * 1024):
        f.write(b"x")
```

One argument differs. Python documents it: "Pass 0 to switch buffering off (only allowed in binary mode)" [S6].

[S6] Built-in Functions, `open()` — Python documentation. <https://docs.python.org/3/library/functions.html#open>

# Two programs, two access patterns

## Sequential access

Each write continues where the last one ended.

## Random access

Each write follows a move to a new offset.

## Why it matters

Lampson advised batch processing in 1983. He named sequential access in the next sentence, as a second and separate reason [S12].

[S12] Hints for Computer System Design — Lampson (1983), p. 19. <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>

# Predict before we run

Write down a ratio. How much slower is the unbuffered program?

**1×**

nothing worth measuring

**10×**

an optimisation

**100×**

a different kind of thing

Writing a number down before the run changes what you do with the result. Taber calls the method predict-observe-explain, and reports that "making a prediction will give students some sense of ownership in the following activity" [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

# The measurement runs here, not on a slide

I print no ratio here, on purpose. A benchmark you did not watch me produce is the kind of claim this course teaches you to distrust.

## Held fixed

Loop, interpreter, iteration count, bytes, filesystem.

## Varied

One argument, `buffering=0` [S6].

## Write down

Both times, the Python version, the buffer size.

## What it includes

The operating system's own memory for file data, in both runs. Name the cache layer your number includes, because you cannot remove all of them [V19].

# Your laptop will not agree with this projector

The buffer size is not a constant. It grew sixteen-fold in Python 3.14.

## Python 3.13

The default "was set to 8192 since 16 years ago" [S7].

## Python 3.14

"the buffer will typically be 128 kilobytes long" [S6].

## Why

Write performance improved "3 to 5 times" over the 8192-byte default [S7]. The reporter measured it. No independent reproduction.

## So

Report your version and buffer size with every number.

[S6] Python documentation, `open()`. [S7] CPython issue 117151. <https://github.com/python/cpython/issues/117151>

# Where does the difference come from?

---

One answer, stated immediately, and then the instrument that proves it.

# Where does the difference come from?

The difference is the number of times each program crosses into the operating system.

Your theory course defined a system call. This lecture counts them: each one makes a **crossing** out of your program, "a privilege transition, not a jump" [V15].

|          |                         |                                           |
|----------|-------------------------|-------------------------------------------|
| <b>A</b> | <b>Unbuffered</b>       | One call per byte, so 1,048,576 calls.    |
| <b>B</b> | <b>Buffered</b>         | One mebibyte divided by the buffer size.  |
| <b>=</b> | <b>So measure first</b> | At 8 KiB, 128 calls. At 128 KiB, 8 calls. |

## What one crossing costs

**4,972**

instructions, one stat call

**13,585**

cycles for the same call

**2,559**

L3 lines evicted — your data, not the kernel's

Soares and Stumm measured this on 2010 hardware [S16]. The counts are not current [V2]. What carries over is the regime distinction, not the numbers. That distinction is a call rate. The lost cache state dominates at one call per 2,000 to 100,000 instructions. The transition cost takes over above that rate [S16].

[S16] FlexSC — Soares and Stumm, OSDI 2010. [https://www.usenix.org/legacy/event/osdi10/tech/full\\_papers/Soares.pdf](https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf)

# What one trace line says

A trace records the system calls a program made, in order, with arguments and return values.

```
open("/dev/null", O_RDONLY) = 3
```

- 1 The name** Which system call ran [S1].
- 2 The arguments** In parentheses, decoded [S1].
- 3 The return value** After the equals sign [S1].

A trace is not a log. A log carries what the program's author chose to print. A trace carries what the program did.

[S1] `strace(1)` — Linux man-pages, which documents this example line. <https://man7.org/linux/man-pages/man1/strace.1.html>

# Count the crossings

The same two programs, run again under a counting tracer.

|                  |                                                               |
|------------------|---------------------------------------------------------------|
| <b>Command</b>   | <code>strace -c -e trace=write python3 a.py</code> [S1]       |
| <b>Program A</b> | One write per byte.                                           |
| <b>Program B</b> | One write per buffer fill.                                    |
| <b>First</b>     | Print <code>io.DEFAULT_BUFFER_SIZE</code> , then divide [S6]. |
| <b>Two runs</b>  | A traced run cannot be timed [V17].                           |

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>

# The count ratio over-predicts the time

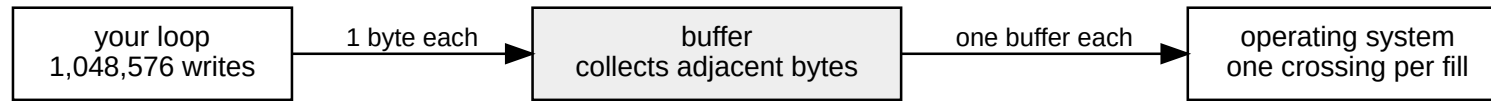
Illustration, not a measurement from this room: a count ratio of 100,000 beside a time ratio of 40 is not a mistake.

$\text{time} \approx \text{crossings} \times \text{fixed cost} + \text{bytes} \times \text{marginal cost [V1]}$

- 1 Two terms** Not a function of bytes alone [V1].
- 2 One term is shared** Both programs write the same mebibyte through the same loop.
- 3 So counts over-predict** Under this model the count ratio exceeds the time ratio.

# What a buffer actually does

A buffer collects adjacent bytes and hands them down in one call.



Cost has two terms: the number of crossings, and the number of bytes [V1]. A buffer cuts the first and leaves the second alone, so the saving sits entirely in the call count.

# Why a seek gives the saving back

A buffer holds bytes for consecutive positions. Move elsewhere and those bytes cannot stay in it.

- |          |                    |                                               |
|----------|--------------------|-----------------------------------------------|
| <b>1</b> | <b>Constraint</b>  | Buffered bytes belong at the old offset.      |
| <b>2</b> | <b>Consequence</b> | Every seek forces the buffer out.             |
| <b>3</b> | <b>Result</b>      | The buffered run converges on the unbuffered. |

The rule is **buffering is fast when writes are adjacent, and buys little when they are not**. Lampson gave the batching half in 1983 [S12].

# Does the seek itself cost a crossing?

Yes. And a pair of system calls exists to remove it.

## Naive

`lseek` then `write` — two crossings for one operation.

## Merged

`pread` and `pwrite` "combine[s] file access and seek operations into a single command" [V3].

## Measured

The merged pair cut the call count [V3]. For the same data volume the study also reports "a relatively reduced load in terms of overall duration" [V27].

# Trust the counts. Do not trust the times.

The tracer stops your program twice per call. That destroys the timing and leaves the counting intact.

## EVERY CALL STOPS

[V13]

So "the counts are trustworthy and the absolute times are not".

Counts only

## GREGG'S DD LOOP

[V17]

One byte per call, "runs 442× slower under strace".

His machine

## RE-RUNS

[V31]

"roughly 37× to 136×", other hardware.

Not a bound

The range does not contain Gregg's figure. Each number measures one machine and one workload.

# Windows counts too — differently

NtTrace, the Windows tracer in this course, prints no counting summary. Both systems count for you anyway.

| PLATFORM | WHERE THE COUNT LIVES                        | WHAT THE FIELD IS DEFINED AS                                           |
|----------|----------------------------------------------|------------------------------------------------------------------------|
| Windows  | GetProcessIoCounters fills IO_COUNTERS [S24] | WriteOperationCount — "The number of write operations performed" [S23] |
| Linux    | the file /proc/pid/io [S25]                  | syscw — the "file write" system calls [S25]                            |

`iodemo.py` and `iodemo.c` print their own count when given `--count`. On Windows they call `GetProcessIoCounters` [S24].

[S23] IO\_COUNTERS (winnt.h) — Microsoft Learn. [https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io\\_counters](https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters) [S24]  
GetProcessIoCounters (winbase.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiounters> [S25] `proc_pid_io(5)` — Linux man-pages. [https://man7.org/linux/man-pages/man5/proc\\_pid\\_io.5.html](https://man7.org/linux/man-pages/man5/proc_pid_io.5.html)

# Counting on Windows: ask the kernel

Windows has no `strace -c`. The kernel counts anyway, and one call reads it [S24].

```
def write_operations():
    handle = win32api.GetCurrentProcess()
    return win32process.GetProcessIoCounters(handle)["WriteOperationCount"]

before = write_operations()
with open(path, "wb") as f:
    f.write(b"one\n"); f.write(b"two\n"); f.write(b"three\n")
after = write_operations()
```

- **Why not a tracer.** NtTrace prints no summary; the program asks the kernel [S24].
- **What it does.** Reads `WriteOperationCount` [S23] twice; the difference is what three lines cost.
- **What to expect.** One or two operations, not three — Python buffered them [S6].

[S24] `GetProcessIoCounters` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiounters> [S29] `win32process.GetProcessIoCounters` — PyWin32. [https://mhammond.github.io/pywin32/win32process\\_\\_GetProcessIoCounters\\_meth.html](https://mhammond.github.io/pywin32/win32process__GetProcessIoCounters_meth.html)

# What the counters will not tell you

Three limits, none visible in the count.

## THE LIMIT

## WHAT IT DOES TO YOUR COUNT

|                           |                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------|
| Not one measurement       | Linux counts system calls [S25]. Microsoft says "write operations performed" [S23]. |
| Operations, not crossings | syscw counts only write calls [S25]. It misses a seek.                              |
| Start-up writes count too | The count covers the process, not your loop. Compare two runs.                      |

Part V carries a real Windows trace [S10]. Equal footing is a course promise, not a per-lecture one.

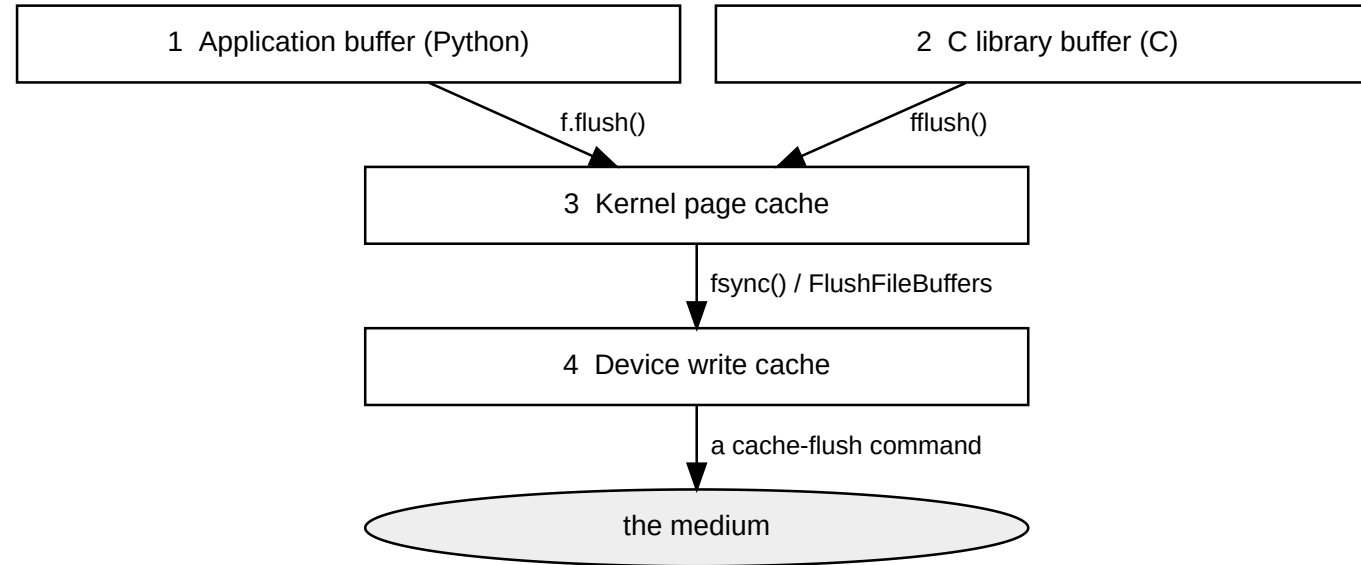
[S23] IO\_COUNTERS (winnt.h) — Microsoft Learn. [https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io\\_counters](https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters) [S25] proc\_pid\_io(5) — Linux man-pages. [https://man7.org/linux/man-pages/man5/proc\\_pid\\_io.5.html](https://man7.org/linux/man-pages/man5/proc_pid_io.5.html) [S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/>

# It is not one buffer — it is a stack

---

We counted the crossings. Now we ask where the bytes actually are.

# It is not one buffer — it is a stack



Levels 1 and 2 are alternative routes into level 3 [V6]. Each arrow is one call [S17] [S3] [S13] [V37] [V33].

[S17] `io.IOBase.flush()` — Python documentation. [S3] `fflush(3)` — Linux man-pages. [S13] `fsync` — The Open Group.

# Four levels, and who empties each one

| LEVEL         | WHO OWNS IT                        | EMPTIED BY                   |
|---------------|------------------------------------|------------------------------|
| 1 Application | the runtime's own buffer [S8]      | <code>f.flush()</code> [S17] |
| 2 C library   | the <code>stdio</code> buffer [S3] | <code>fflush()</code> [S3]   |
| 3 Kernel      | the page cache [V32]               | <code>fsync()</code> [S13]   |
| 4 Device      | the drive's write cache [V20]      | a cache-flush command [V33]  |

The page cache lives in the machine's memory [V32]. It is not the disk's cache — the disk has its own, one level further down [V20].

Reads use the same four levels, in the other direction. The kernel fetches pages ahead of a sequential reader, so "a sequential `read()` just happens to return faster because the data was already fetched" [V34].

# To flush is to cross one boundary

The verb means one thing here: to hand a buffer's contents to the next level down.

The C library manual says it plainly: "Note that fflush() flushes only the user-space buffers provided by the C library. To ensure that the data is physically stored on disk the kernel buffers must be flushed too" [S3].



**Do not say**

Save, commit, persist — all claim durability.



**Say**

Flush. It names one boundary.

[S3] fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

# Wholesale down, retail up

Every level buys a large block from the level below. It sells small pieces to the level above.

That asymmetry is why every level in this diagram holds a buffer. Each level's "unit of purchase is larger than its unit of sale" [V18].

- |          |                             |                                     |
|----------|-----------------------------|-------------------------------------|
| <b>1</b> | <b>Buys in blocks</b>       | 4096 bytes on an ext4 file [V7].    |
| <b>2</b> | <b>Sells in bytes</b>       | Your loop writes one byte.          |
| <b>3</b> | <b>Same shape elsewhere</b> | Allocators, caches, network stacks. |

# One program, one buffer — C and Python take different paths

Each program uses at most one buffer, and not the same one.

```
iodemo.c    fwrite()  →  stdio buffer (level 2)  →  write()  →  page cache  
iodemo.py   f.write() →  BufferedWriter (level 1) →  write()  →  page cache
```

The C program's buffer is the C library's `stdio` [S3] [V7]. Python's is a `BufferedWriter` [S8]. `FileIO` calls `write()` itself, so no `stdio` buffer sits under Python on either platform [S26] [S27] [S28] [V6].

The four levels are the union of both paths, not one program's. Every program crosses levels 3 and 4, and at most one of levels 1 and 2.

Python documents [128 KiB](#) as typical [S6]. On one ext4 file the C library took 4096 bytes, its block size [V7].

[S6] `open()` — Python documentation. <https://docs.python.org/3/library/functions.html#open>

# The bottom level acknowledges early too

Level four is the uncomfortable one, and skipping it would make the diagram dishonest.

- 1 The device holds a buffer** "most HDDs and many SSDs acknowledge a write once it lands in an onboard DRAM cache" [V20].
- 2 The standard allows a no-op** On fsync: "a null implementation is permitted" [S13].
- 3 Software cannot check** No trace separates a device that flushed from one that only reported it [V5].

[S13] fsync — The Open Group. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

# Three lines that print nothing

A demonstration. You have the stack now, so predict this one.

```
import os  
print("hello")  
os._exit(0)
```

**To a terminal**

Does hello appear?

**Redirected to a file**

What does the file contain?

Write down both answers before I run either one [S14].

[S14] Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>

# Why the file is empty

The rule sits in the C standard, not in the kernel.

- 1 Mode follows destination**

Output is "fully buffered if and only if the stream can be determined not to refer to an interactive device" [S5].
- 2 A terminal is interactive**

Bytes go out per line.
- 3 A file is not**

Bytes wait.
- 4 The exit skips cleanup**

No write reaches the kernel.

[S5] C99 draft §7.19.3. <https://busybox.net/~landley/c99-draft.html>

# What this demonstration does not prove

It proves level one is real. It proves nothing about levels three and four.

"the honest classroom demo is a power-cut, not a kill -9. Killing the process loses only the *runtime's* buffer; the kernel still writes back." [V14]

- |          |                         |                                                                   |
|----------|-------------------------|-------------------------------------------------------------------|
| <b>1</b> | <b>What died</b>        | The process, and the level-one buffer with it.                    |
| <b>2</b> | <b>What survived</b>    | Anything already in the page cache [V14].                         |
| <b>3</b> | <b>Wrong conclusion</b> | "So <b>flush() was enough.</b> " It was enough against this test. |

# Which levels can you actually watch?

I promised three boundaries of four. Here is the tool each one needs.

|                  |       |                                                    |              |
|------------------|-------|----------------------------------------------------|--------------|
| APP → LIBRARY    | [V10] | "printf is visible to ltrace and write to strace". | Another tool |
| LIBRARY → KERNEL | [S1]  | Where our tracer attaches.                         | Today        |
| KERNEL → DEVICE  | [V30] | Block-layer tracing, not in this course.           | Elsewhere    |
| DEVICE → MEDIUM  | [V5]  | No trace separates a flush from a report of one.   | Never        |

# The same write, at three levels

---

Now that the levels have a reason to exist, walk down them with one small program.

# The same write, at three levels

| LEVEL       | PYTHON                           | C                             | BUFFERED? |
|-------------|----------------------------------|-------------------------------|-----------|
| Application | <code>f.write(b"x")</code> [S6]  | —                             | yes [S6]  |
| C library   | —                                | <code>fwrite(...)</code> [S3] | yes [S3]  |
| System call | <code>os.write(...)</code> [S20] | <code>write(...)</code> [S2]  | no        |

The bottom row takes a **file descriptor** — the small integer naming an open file. You saw one already: the `= 3` in the trace line was a file descriptor [S1].

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S2] `write(2)` — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S3] `fflush(3)` — Linux man-pages. [S6] Python documentation, `open()`. [S20] `os` — Miscellaneous operating system interfaces, `os.write()`. <https://docs.python.org/3/library/os.html#os.write>

# Meet the program: `iodemo.c`

Nine lines of C. The first open fails on purpose, so the trace shows an error path.

```
f = fopen("no-such-file.txt", "rb"); /* 1. deliberate failure */
if (f != NULL) { fclose(f); return 2; }

f = fopen(path, "wb"); /* 2. the real file */
if (f == NULL) { return 1; }

fwrite("one\n", 1, 4, f); /* 3. three writes */
fwrite("two\n", 1, 4, f); /*    fourteen bytes */
fwrite("three\n", 1, 6, f);

if (fclose(f) != 0) { return 3; } /* 4. flush, then close */
```

It prints nothing, on purpose. A silent program makes a legible trace.

# Three calls in the source. One in the trace.

# 3

`fwrite` calls in the source

# 14

bytes written in total

# 1

`write` call in the trace

The three `fwrite` calls go into one C library buffer. The manual calls that level "the user-space buffers provided by the C library" [S3]. The line to look for in the live trace, predicted from the source:

```
write(3, "one\ntwo\nthree\n", 14) = 14
```

The 3 is the file descriptor. The 14 is all three `fwrite` calls, in one crossing.

[S3] `fflush(3)` — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>

# fwrite takes a size and a count

Four arguments, and two of them multiply.

```
fwrite("three\n", 1, 6, f);  
/*      data      ^  ^  stream  
               |  +-- number of items  
               +----- size of one item, in bytes */
```

## 1 It returns a count

Items written, not bytes. The count equals bytes only when the item size is 1 [S21].

## 2 Fewer is legal

Below it, "a successful write() may transfer fewer than count bytes" [S2].

[S2] write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html> [S21] fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>

# Two operating systems, side by side

---

Read as a comparison. Running the tools is the first lab's job.

# Why NtTrace and not Process Monitor

Two Windows tools, two layers. One answers this lecture's question.

## PROCESS MONITOR

- 1 Shows "real-time file system, Registry and process/thread activity" [S18]
- 2 The usual Windows choice
- 3 The wrong layer for us

## NTTRACE

- 4 Reports native API calls, in user mode [S10]
- 5 Runs on the debugger interface, so it "needs no driver and no administrator rights" [V21]

[S18] Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

# Three Windows words you will need

## Win32

The documented interface a Windows program calls. The 32 is history, not a width. Microsoft renamed it the Windows API. The new name "more accurately reflects its roots in 16-bit Windows and its support on 64-bit Windows" [S22].

## The native API

The layer below Win32, inside `ntdll.dll`. Microsoft calls it "internal to the operating system" [S9].

## The object manager

The kernel component that names open objects. Its paths begin `\\??\\` [V22].

[S9] `winternl.h` — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation> [S22]  
Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>

# A real Windows trace

You can already read this. The anatomy matches the Linux line, and the vocabulary differs.

```
NtOpenFile(FileHandle=0x12fb38 [0x14], DesiredAccess=FILE_EXECUTE|SYNCHRONIZE,  
  ObjectAttributes="\??\C:\WWW\NtTrace\", IoStatusBlock=0x0012FAE4 [0/1],  
  ShareAccess=3, OpenOptions=0x21) ⇒ 0  
NtQueryAttributesFile(ObjectAttributes="\??\C:\WINDOWS\system32\cmd.exe.Local",  
  Attributes=0x0012FADC [0]) ⇒ 0xc0000034 [2 'The system cannot find the file specified.']  
...  
NtWriteFile(FileHandle=4, Event=0, ApcRoutine=null, ApcContext=null,  
  IoStatusBlock=0x0012FD8C [0/0x29], Buffer=0x4ad30e40, Length=0x29,  
  ByteOffset=null, Key=null) ⇒ 0
```

Three lines excerpted from Roger Orr's published trace of cmd [S10]. Ignore every argument except the handle, the path and the return value.

[S10] NtTrace — Roger Orr, project page. <https://rogerorr.github.io/NtTrace/>

# A handle is not a file descriptor

First difference, visible in the first argument of almost every line.

## LINUX

A file descriptor. A small integer indexing a per-process table.  
The trace showed = 3 [S1].

## WINDOWS

A handle. An opaque value naming an open object. The trace shows `FileHandle=4` [S10].

Same job in a trace, different mechanism in the system. Do not use one word for the other.

[S1] `strace(1)` — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html> [S10] NtTrace — Roger Orr. <https://rogerorr.github.io/NtTrace/>

# Two error namespaces in one return value

Second difference, and it is the layer boundary showing up where nobody expects it.

```
⇒ 0xc0000034 [2 'The system cannot find the file specified.']
```

- |          |                      |                                             |
|----------|----------------------|---------------------------------------------|
| <b>1</b> | <b>First number</b>  | An NTSTATUS, the native layer's code [S10]. |
| <b>2</b> | <b>Second number</b> | The Win32 error it translates into [S10].   |
| <b>3</b> | <b>Linux</b>         | One errno. Nothing to translate.            |

[S10] NtTrace — Roger Orr, published sample trace. <https://rogerorr.github.io/NtTrace/>

# The stable boundary sits at a different level

Third difference, and the other two are symptoms of it.

The Windows path reads "\\??\\C:\\WINDOWS\\system32\\cmd.exe.Local". That prefix "is the object-manager namespace, which a Win32 program never types" [V22].

## LINUX

[V8]

The system call interface is the public contract.

Syscall

## WINDOWS

[S9]

Native entry points are "subject to change from one release of Windows to another".

Win32

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

# Where the boundary sits is contested history

Two readings survive the evidence. I will not pick one for you.

- 1 Reading A: deliberate** POSIX, OS/2 and Win32 each reached NT through the native API [S19].
- 2 Reading B: vestige** A reported POSIX subsystem that was "never really completed" [S15].
- 3 Linux had no choice** It "had to make the kernel boundary itself the stable interface" [V38].

[S15] The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/> [S19] Inside Native Applications — Russinovich (2006). <https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>

# Linux is the outlier, not Windows

The tempting conclusion is backwards, and the evidence is a list of other systems.

"Linux is the only mainstream system that treats the raw syscall interface as a public contract." [V9]

- |          |                                 |                                                                   |
|----------|---------------------------------|-------------------------------------------------------------------|
| <b>1</b> | <b>Who declines the promise</b> | FreeBSD, OpenBSD, NetBSD, illumos, macOS — and not equally [V36]. |
| <b>2</b> | <b>Why they can</b>             | Each ships its own C library [V9].                                |
| <b>3</b> | <b>So say different</b>         | Never say worse.                                                  |

# What that trace cannot do

The honest caption, so nothing here is stronger than the evidence.

- 1 Not a matched pair** "It is not a matched pair." It follows the shell, not our program [V23].
- 2 Not what you will see** A 32-bit, XP-era trace, not today's addresses [V35].
- 3 One maintainer** And the layer it watches changes between releases [S9].
- 4 What survives** The handle, the path, the two error namespaces.

[S9] winternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>

# What we have not looked at

---

One question I will not answer, and the work that follows from it.

# **write() returned success. The bytes are not on a disk.**

The whole hour built towards this sentence. I leave it unresolved.

The manual says it plainly: "A successful return from write() does not make any guarantee that data has been committed to disk" [S2].

Saltzer, Reed and Clark gave the general form in 1984. A lower level may offer "an incomplete version of the function... useful as a performance enhancement" [S11].

**So when is it actually safe?** That is the durability lecture's question, and this lecture leaves it open.

[S2] write(2) — Linux man-pages. [S11] End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984).  
<https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

# Buffering moves your errors

The error arrives somewhere other than where you caused it.

- 1**      **Why it moves**      The bytes only reached a buffer.
- 2**      **Where it arrives**      Where the library flushes, so at `fclose`.
- 3**      **The rule**      Check the return value of the call that flushes.
- 4**      **The idiom hides it**      "the most common durability idiom in every mainstream language... contains no durability step anywhere" [V4].

# Never retry a failed close

The obvious repair is wrong. Three storage engines dropped the retry in 2018 [V40].

"Retrying the close() after a failure return is the wrong thing to do, since this may cause a reused file descriptor from another thread to be closed" [S4].

Three databases reached the same conclusion one level down, for fsync: PostgreSQL, MySQL's InnoDB and MongoDB's WiredTiger [V40]. WiredTiger wrote the reason into its own source comment: "we don't retry fsync-style calls and panic if a flush operation fails" [V24].

[S4] close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>

# Networking and IPC are this same mechanism

Not new subjects. The same objects and the same calls, wearing different names.

- 1 A socket on Linux** It is a file descriptor, and Linux makes that contract binding [V28].
- 2 A socket on Windows** An opaque SOCKET, and Windows makes Winsock the contract [V28].
- 3 A pipe** A file descriptor too, so the same calls reach it [V29].

Depth belongs to the next lecture. The job here is to stop you filing networking under "something else".

# Honest limits

What today's tools cannot see.

- **The device level** [V5]  
Not observable from software: no trace can check it.
- **Asynchronous I/O** [V11]  
The tracer sees the setup, then no I/O.

# Honest limits, continued

And what today's tools cost.

- **strace in production**

[V25]

Even the cheaper `--seccomp-bpf` mode [V39] "should therefore not be used on production systems!"

- **Mapping, crash dumps**

Each has its own lecture.

# What to do on Monday

- |          |                                              |                                           |
|----------|----------------------------------------------|-------------------------------------------|
| <b>1</b> | <b>Assignment 1 measures<br/>takeaway 1</b>  | Two settings, two patterns, one mebibyte. |
| <b>2</b> | <b>Start from section one</b>                | Add the random-access variant.            |
| <b>3</b> | <b>Report your buffer size</b>               | Beside every number.                      |
| <b>4</b> | <b>Do not benchmark under the<br/>tracer</b> | Count traced, time untraced [V17].        |
| <b>5</b> | <b>Count in the dev container</b>            | Both platforms count there.               |
| <b>6</b> | <b>Week 2 opens</b>                          | On 22 September.                          |

# Summary — what follows

- |   |                                 |                                                         |
|---|---------------------------------|---------------------------------------------------------|
| 1 | <b>Ask "accepted or done?"</b>  | A fast return reports acceptance, not completion [V12]. |
| 2 | <b>Count, do not reason</b>     | No level reports which one holds your data [V16].       |
| 3 | <b>Name the level</b>           | Your buffer, the C library, the page cache, the device. |
| 4 | <b>Measure your own machine</b> | Never quote a remembered constant [S7].                 |

# References • 1/11

- **[S1]** strace(1) — Linux man-pages. <https://man7.org/linux/man-pages/man1/strace.1.html>
- **[S2]** write(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/write.2.html>
- **[S3]** fflush(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fflush.3.html>
- **[S4]** close(2) — Linux man-pages. <https://man7.org/linux/man-pages/man2/close.2.html>
- **[S5]** C99 draft, §7.19.3. <https://busybox.net/~landley/c99-draft.html>
- **[S6]** Built-in Functions — open() — Python documentation.  
<https://docs.python.org/3/library/functions.html#open>
- **[S7]** Increase io.DEFAULT\_BUFFER\_SIZE — CPython issue 117151.  
<https://github.com/python/cpython/issues/117151>

# References · 2/11

- **[S8]** PEP 3116 — New I/O (2007). <https://peps.python.org/pep-3116/>
- **[S9]** wininternl.h — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winternl/nf-winternl-ntquerysysteminformation>
- **[S10]** NtTrace — Native API tracing for Windows — Roger Orr. <https://rogerorr.github.io/NtTrace/>
- **[S11]** End-to-End Arguments in System Design — Saltzer, Reed and Clark (1984). <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>
- **[S12]** Hints for Computer System Design — Lampson (1983). <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/acrobat-17.pdf>
- **[S13]** fsync — The Open Group Base Specifications. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/fsync.html>

# References • 3/11

- **[S14]** Predict-Observe-Explain — Taber. <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S15]** The sad history of the Microsoft POSIX subsystem — Reiter (2010). <https://brianreiter.org/2010/08/24/the-sad-history-of-the-microsoft-posix-subsystem/>
- **[S16]** FlexSC: Flexible System Call Scheduling with Exception-Less System Calls — Soares and Stumm, OSDI 2010. [https://www.usenix.org/legacy/event/osdi10/tech/full\\_papers/Soares.pdf](https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Soares.pdf)
- **[S17]** io.IOBase.flush() — Python documentation. <https://docs.python.org/3/library/io.html#io.IOBase.flush>
- **[S18]** Process Monitor — Sysinternals, Microsoft Learn. <https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

# References • 4/11

- **[S19]** Inside Native Applications — Russinovich, Microsoft Learn (2006).  
<https://learn.microsoft.com/en-us/sysinternals/resources/inside-native-applications>
- **[S20]** os — Miscellaneous operating system interfaces, os.write() — Python documentation.  
<https://docs.python.org/3/library/os.html#os.write>
- **[S21]** fread(3), which documents fwrite(3) — Linux man-pages. <https://man7.org/linux/man-pages/man3/fwrite.3.html>
- **[S22]** Windows API index — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list>
- **[S23]** IO\_COUNTERS (winnt.h) — Microsoft Learn. [https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io\\_counters](https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-io_counters)

# References · 5/11

- **[S24]** GetProcessIoCounters (winbase.h) — Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getprocessiounters>
- **[S25]** proc\_pid\_io(5) — Linux man-pages. [https://man7.org/linux/man-pages/man5/proc\\_pid\\_io.5.html](https://man7.org/linux/man-pages/man5/proc_pid_io.5.html)
- **[S26]** Modules/\_io/fileio.c — CPython. [https://github.com/python/cpython/blob/main/Modules/\\_io/fileio.c](https://github.com/python/cpython/blob/main/Modules/_io/fileio.c)
- **[S27]** Python/fileutils.c — CPython. <https://github.com/python/cpython/blob/main/Python/fileutils.c>
- **[S28]** Low-level I/O — Microsoft Learn, C runtime library. <https://learn.microsoft.com/en-us/cpp/c-runtime-library/low-level-i-o>

# References · 6/11

- **[S29]** win32process.GetProcessIoCounters — PyWin32, Mark Hammond.  
[https://mhammond.github.io/pywin32/win32process\\_\\_GetProcessIoCounters\\_meth.html](https://mhammond.github.io/pywin32/win32process__GetProcessIoCounters_meth.html)
- **[V1]** A system call is not a function call — the two-term cost model.
- **[V2]** A system call is not a function call — the measured footprint of one crossing.
- **[V3]** A system call is not a function call — pread and pwrite merge the seek.
- **[V4]** A Successful write() Is a Handoff, Not a Save — the idiom with no durability step.
- **[V5]** A Successful write() Is a Handoff, Not a Save — the device level is unobservable.
- **[V6]** Standard output is buffered differently depending on where it points — CPython above libc.

# References · 7/11

- **[V7]** Standard output is buffered differently depending on where it points — the measured 4096-byte buffer.
- **[V8]** Windows and Linux place the stable syscall boundary at different layers — the nomination argument.
- **[V9]** Windows and Linux place the stable syscall boundary at different layers — Linux as the outlier.
- **[V10]** The System Call Boundary — ltrace against strace.
- **[V11]** The System Call Boundary — io\_uring with a polling thread.
- **[V12]** Where the Abstraction Leaks — accepted, not done.
- **[V13]** Where the Abstraction Leaks — trust the counts, not the times.

# References · 8/11

- **[V14]** Where the Abstraction Leaks — a kill is not a power cut.
- **[V15]** Where the Abstraction Leaks — a privilege transition, not a jump.
- **[V16]** Where the Abstraction Leaks — count, do not deduce.
- **[V17]** Observing a Running System — the 442-times worst case.
- **[V18]** The Memory Hierarchy from Python to Physical Pages — wholesale down, retail up.
- **[V19]** Virtual Memory and the Page Cache — name the cache level in every number.
- **[V20]** File IO and Durability — the device acknowledges early.
- **[V21]** NtTrace — Native API tracing for Windows (Roger Orr) — a debugger, not a driver.
- **[V22]** NtTrace — Native API tracing for Windows (Roger Orr) — the object-manager path prefix.

# References · 9/11

- **[V23]** NtTrace — Native API tracing for Windows (Roger Orr) — what the published trace cannot do.
- **[V24]** PostgreSQL's fsync() surprise — never retry after fsync fails.
- **[V25]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — not for production.
- **[V26]** The System Call Boundary — the calls a tracer cannot see.
- **[V27]** A system call is not a function call — what merging the seek did to overall duration.
- **[V28]** Sockets and Network I-O — a Linux socket is a file descriptor, a Windows socket is not.
- **[V29]** Sockets and Network I-O — sockets, pipes and terminals are all file descriptors.
- **[V30]** File IO and Durability — block-layer tracing watches the kernel-to-device boundary.

# References • 10/11

- **[V31]** Observing a Running System — the published range for the same benchmark, on other machines.
- **[V32]** Virtual Memory and the Page Cache — the page cache is the machine's own memory.
- **[V33]** File IO and Durability — the kernel issues the device cache-flush command.
- **[V34]** Virtual Memory and the Page Cache — readahead moves data up the same four levels.
- **[V35]** NtTrace — Native API tracing for Windows (Roger Orr) — the published trace is 32-bit and XP-era.
- **[V36]** Windows and Linux place the stable syscall boundary at different layers — the five systems do not decline the promise equally.
- **[V37]** File IO and Durability — FlushFileBuffers is the Windows counterpart of fsync.

# References • 11/11

- **[V38]** Windows and Linux place the stable syscall boundary at different layers — why Linux could not move its own boundary.
- **[V39]** Every Tracing Mechanism Trades Observation Cost for a Blind Spot — what the cheaper seccomp-bpf mode costs.
- **[V40]** PostgreSQL's fsync() surprise — three engines dropped the retry in 2018.
- **[V41]** PostgreSQL's fsync() surprise — the three engines are independent of each other.

The handout PDF carries the full reference list, and a one-page summary of every [V] source with its own external references.

TAKE IT WITH YOU

# Everything from this talk

---

[Slides \(PDF\)](#)

[Handout \(PDF\)](#)

[Speaker notes \(PDF\)](#)

[Everything \(.zip\) — readable by an AI agent](#)

[ondrasek.github.io/talks](https://ondrasek.github.io/talks)

[me@ondrejkrasicek.com](mailto:me@ondrejkrasicek.com)